8.3 Docker部署与生产配置

> **生成模型**：gpt-5.4 (openai/gpt-5.4)
> **Token 消耗**：输入 ~17k tokens，输出 ~4.2k tokens（估算，本节）

如果你准备把 OpenClaw 稳定跑在服务器上，Docker 基本是最省心的方案。原因很简单：环境一致、迁移方便、升级清楚，回滚也不至于手忙脚乱。尤其是在中国大陆环境里，镜像源、依赖安装、系统包差异都可能影响结果，容器化能帮你把这些问题压小很多。

这一节不深挖源码，只从部署视角看三件事：Dockerfile 在干嘛、 `docker-compose.yml` 怎么配、生产环境还要补哪些安全和持久化设置。

8.3.1 先理解 Dockerfile：它不是魔法，就是一套可重复安装脚本

从部署角度看，OpenClaw 的 Dockerfile 有几个关键动作：

1. 选一个 Node 22 的基础镜像。
2. 安装构建用到的工具，比如 Bun。
3. 先复制依赖描述文件，再安装依赖，利用层缓存。
4. 复制源码并构建项目。
5. 切到非 root 用户运行。
6. 默认启动 Gateway。

把它简化以后，大概可以理解成这样：

```dockerfile
FROM node:22-bookworm

RUN corepack enable
WORKDIR /app

COPY package.json pnpm-lock.yaml pnpm-workspace.yaml ./
RUN pnpm install --frozen-lockfile

COPY . .
RUN pnpm build && pnpm ui:build

ENV NODE_ENV=production
USER node
CMD ["node", "openclaw.mjs", "gateway", "--allow-unconfigured"]
```

这里有几个点值得你记住。

为什么不是 `alpine`

很多人看到 Docker 第一反应就是追求镜像小，想换成 Alpine。但 OpenClaw 这类项目常常依赖更完整的系统库和原生模块， `bookworm` 这类 Debian 基础镜像更稳。镜像大一点，换来的是构建少踩坑，这笔账通常划算。

为什么要先复制依赖文件

这是 Docker 层缓存最经典的用法。只要 `package.json` 和锁文件没变，重新构建时就不必重新安装所有依赖。你改了几行业务配置，不需要每次都把整个依赖树重下一遍。

为什么要用非 root 用户

这不是“高级玩法”，而是生产环境基本常识。容器不是绝对安全边界，但把服务放到普通用户下运行，至少能减少很多低级风险。

8.3.2 在中国大陆构建镜像时要注意什么

Docker 在国内最容易卡的，往往不是容器本身，而是拉基础镜像、拉 npm 包、安装系统依赖。你如果直接用默认源，构建速度可能非常难看。

Docker 镜像加速

不同云厂商都提供镜像加速器，通常可以把 Docker 的 `daemon.json` 配成这样：

```
{
  "registry-mirrors": [
    "https://your-mirror.example.com"
  ]
}
```

重启 Docker：

```
sudo systemctl daemon-reload
sudo systemctl restart docker
```

npm 和 pnpm 镜像

```
npm config set registry https://registry.npmmirror.com
pnpm config set registry https://registry.npmmirror.com
```

构建命令

```
docker build -t openclaw:local .
```

如果你有额外系统依赖，也可以通过构建参数传进去：

```
docker build \
  --build-arg OPENCLAW_DOCKER_APT_PACKAGES="ffmpeg git curl" \
  -t openclaw:local .
```

8.3.3 `docker-compose.yml` 怎么看

很多人第一次看 Compose 文件，会觉得条目很多。其实你就盯住这几块：镜像、端口、环境变量、卷挂载、重启策略。

一个适合生产起步的示例可以写成这样：

```yaml
services:
  openclaw-gateway:
    image: openclaw:local
    container_name: openclaw-gateway
    restart: unless-stopped
    env_file:
      - ./env.production
    ports:
      - "127.0.0.1:8945:8945"
    volumes:
      - ./data:/home/node/.openclaw
      - ./logs:/app/logs
      - /var/run/docker.sock:/var/run/docker.sock
    command: ["node", "openclaw.mjs", "gateway", "--allow-unconfigured"]

  openclaw-sandbox:
    image: openclaw-sandbox:local
    container_name: openclaw-sandbox
    restart: unless-stopped
    profiles: ["sandbox"]
```

这个文件里，最重要的是下面几项。

restart: unless-stopped

服务器重启后，容器会自动拉起来。对长期运行服务来说，这项很有用。

127.0.0.1:8945:8945

这表示只把端口绑定到本机回环地址，而不是直接暴露到公网。生产环境更推荐让 Nginx 反代到它，而不是让外网直接打容器端口。

env_file

把敏感配置放到独立文件里，别全塞进 Compose 里。这样你换环境也更清楚。

volumes

这项特别重要。没有卷挂载，容器一删，很多状态就没了。

8.3.4 环境变量怎么配才不乱

OpenClaw 的生产配置最好拆成三层：

1. `openclaw.json` ： 结构化主配置。
2. `env.production` ： 密钥、Token、Secret。
3. Compose 文件：运行参数和卷挂载。

一个 `env.production` 示例：

```
OPENCLAW_GATEWAY_TOKEN=replace-me
OPENCLAW_LOG_LEVEL=info
OPENCLAW_BASE_URL=https://bot.example.cn
OPENCLAW_DATA_DIR=/home/node/.openclaw
QWEN_API_KEY=replace-me
FEISHU_APP_SECRET=replace-me
DINGTALK_CLIENT_SECRET=replace-me
WECOM_AGENT_SECRET=replace-me
```

启动前先检查有没有漏变量：

```
grep -n 'replace-me' env.production
```

正式环境里，建议不要把 `.env` 提交到仓库；也不要把真正的生产密钥放进镜像里。镜像应该是可复制的，密钥应该是运行时注入的。

8.3.5 哪些目录一定要做持久化

OpenClaw 不是那种“纯 stateless”的服务。至少下面几类数据，建议挂卷保留：

- 配置文件
- 会话数据
- 日志
- 媒体缓存或下载文件
- 技能、插件、本地工作目录

一个更完整的挂载示例：

```yaml
volumes:
  - ./persist/config:/home/node/.openclaw
  - ./persist/logs:/app/logs
  - ./persist/workspaces:/workspaces
  - ./persist/extensions:/app/extensions-local
```

为什么这一步重要？因为你后面升级镜像、替换主机、回滚版本，都要靠这些数据活下来。没有持久化，容器升级一次，配置和状态可能就重新开始了。

8.3.6 Docker Sandbox 为什么值得单独讲

OpenClaw 里有一类很敏感的能力：工具执行。尤其是 Bash、文件系统、浏览器自动化这类东西，一旦你允许 Agent 直接操作系统环境，安全问题就不能只靠“我相信模型不会乱来”。

Docker sandbox 的思路，就是把高风险执行面关进更小的隔离空间里。你可以把它理解成一层额外保险。

一个简化理解

```
主 Gateway 容器：负责会话、路由、平台接入
Sandbox 容器：负责高风险工具执行
宿主机：只暴露必要的卷和 Docker 能力
```

生产环境里为什么要开

- 降低工具执行直接碰宿主机的风险。
- 多个会话之间更容易隔离。
- 出问题时更容易清理和重建。

当然，它不是银弹。你把 `docker.sock` 整个暴露给容器，安全边界还是会被拉薄。所以生产里要看清楚：你到底是为了方便，还是为了真的隔离。

8.3.7 一个更接近生产的 Compose 样板

```yaml
services:
  openclaw-gateway:
    image: openclaw:local
    restart: unless-stopped
    env_file:
      - ./env.production
    ports:
      - "127.0.0.1:8945:8945"
    volumes:
      - ./persist/config:/home/node/.openclaw
      - ./persist/logs:/app/logs
      - ./persist/workspaces:/workspaces
    healthcheck:
      test: ["CMD", "curl", "-f", "http://127.0.0.1:8945/health"]
      interval: 30s
      timeout: 5s
      retries: 3
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "5"

  nginx:
    image: nginx:stable
    restart: unless-stopped
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx/openclaw.conf:/etc/nginx/conf.d/default.conf:ro
      - ./certs:/etc/nginx/certs:ro
    depends_on:
      - openclaw-gateway
```

这里多了三样很有价值的东西：

- `healthcheck`： 知道服务是不是活着。
- `logging`： 控制日志文件别无限长。
- `nginx`： 把公网入口和应用容器分开。

8.3.8 常用命令别只记一个 `up -d`

下面这些命令，生产环境里都很常用：

```
# 启动
docker compose up -d

# 看状态
docker compose ps

# 看日志
docker compose logs -f openclaw-gateway

# 重建并启动
docker compose up -d --build

# 停止
docker compose down

# 查看卷
docker volume ls
```

如果你要升级版本，一个比较稳的习惯是：

```
docker compose pull

docker compose up -d

docker compose logs -f --tail=100
```

先更新，再盯日志，不要更新完就走。

8.3.9 生产检查清单

最后给你一份非常实用的检查清单。上线前，逐项过一遍：

- 镜像能在国内网络环境下稳定构建或拉取。
- Compose 文件里没有把内部端口直接暴露到公网。
- 所有 Secret 都通过环境变量注入，没有硬编码进镜像。
- 配置、日志、工作目录都已经挂卷持久化。

- Nginx 和 HTTPS 已配置完成。
- 域名已经备案，至少不会卡在正式上线前夜。
- 模型服务在大陆环境下访问稳定。
- Sandbox 策略已开启，或者你清楚知道自己为什么没开。
- 日志滚动、健康检查、自动重启已配置。
- 升级和回滚步骤已经演练过一次。

如果你愿意再多做一步，建议加上备份：

```
tar -czf backup-$(date +%F).tar.gz ./persist ./env.production ./docker-compose.yml
```

别等故障发生以后才想起要备份。那时候通常已经晚了。

本节小结

Docker 部署 OpenClaw 的核心，不在于把容器跑起来，而在于把镜像构建、环境变量、卷挂载、反向代理和 sandbox 一起收拾干净。Dockerfile 解决的是“怎么稳定打包”，Compose 解决的是“怎么长期运行”，生产配置解决的是“出了问题还能不能稳住”。在中国大陆环境下，再额外把镜像源、备案和跨境模型链路考虑进去，这套部署才算真的能用。